Accelerated Filtering on Graphs using Lanczos method

Scientific Computing project report

Alberto Defendi

Abstract

In the following we implement Lanczos and Arnoldi algorithm to study the problem of matrix functions evaluation in the context of graph signal processing. We begin studying and assessing the computational cost of these algorithms and their orthogonalization variants. Then, through a series of experiments, we consider the iteration residual, to relate it with the real error and to study how it affects convergence.

Contents

1 Introduction

We introduce basic graph theory concepts and we recall some theoretical results for the experiment.

1.1 Signal processing on graphs

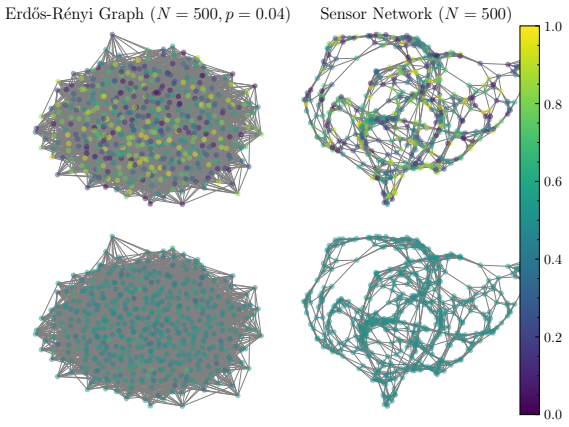
We consider a weighted undirected graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{W})$, where $\mathcal{V} \subset \mathbb{R}^N$ is the set of vertices, $\mathcal{E} \subset \mathbb{R}^M$ is the set of edges, and $\mathcal{W} : \mathcal{V} \times \mathcal{V} \to \mathbb{R}$ is a weight function. We represent the weight function with a $N \times N$

matrix W such that $W_{i,j} = \begin{cases} W(v_i, v_j), & \text{if } (v_i, v_j) \in \mathcal{E} \\ 0, & \text{otherwise} \end{cases}$ for all $i, j = 1, \dots, |\mathcal{V}|$, and for our needs, we assume W to be symmetric, that is $W_{i,j} = W_{j,i}$.

In the study of graph signal processing, we model the idea of sending a signal to a node as assigning a value to a vertex with a function $s : \mathcal{V} \to \mathbb{R}$, whose values can be represented as a vector $s = s(\mathcal{V}) = [s_1, \dots, s_N] \in \mathbb{R}^N$ where each entry $s_i := s(v_i)$ represents the signal sent over a node $v_i \in \mathcal{V}$. We also keep track of the weight of each vertex with a function $d(i) := \sum_{j=1}^{N} W_{i,j}$, and we call $D = \text{diag}(d(1), \dots, d(N))$ the degree matrix.

In this setting, we introduce the graph Laplacian $\mathcal{L}$ defined as $\mathcal{L} = D - W$. By construction, $\mathcal{L}^T = \mathcal{L}$, thus the Lanczos method can be safely applied to our problem. This matrix has some other useful properties that we will list in the following Proposition.

Figure 1: Filtering signal on graphs.



Proposition 1.1. *The Laplacian matrix $\mathcal{L} \in \mathbb{R}^{N \times N}$ satisfies the following properties:*

1. *$\mathcal{L}$ is symmetric and positive semi-definite.*
2. *The smallest eigenvalue of $\mathcal{L}$ is 0 with corresponding eigenvector the constant one vector $\mathbf{1}$.*
3. *$\mathcal{L}$ has N non-negative real-valued eigenvalues $0 = \lambda_0 \leq \lambda_1 \leq \dots \leq \lambda_{N-1}$.*

Proof. See [1]. □

Remark 1.2. In light of this proposition and by the spectral theorem, the Laplacian can be decomposed as:

$$\mathcal{L} = U \Lambda U^*,$$

where $U = [u_0, \dots, u_{N-1}]$ is a $N \times N$ orthogonal matrix and we call it *Fourier basis*, and $\Lambda = \text{diag}(\lambda_0, \dots, \lambda_{N-1})$ is the matrix of its eigenvalues, which are monotonically increasing, and the vectors in U are in the same order that the eigenvalues.

Definition 1.3 (Graph signal)**.** A graph signal is a continuous function $g : \mathbb{R}^+ \to \mathbb{R}$. For our needs, we will assume g to be defined on the spectrum of $\mathcal{L}$.

Diagonalising the Laplacian would give an easy way to compute the function $g(\mathcal{L})$ by evaluating it on the eigenvalues of $\mathcal{L}$, which means computing $g(\mathcal{L}) = U g(\Lambda) U^*$. In matrix notation, applying a filter to a graph $\mathcal{G}$ corresponds to the operation

$$s' := g(\mathcal{L}) s = U g(\mathcal{L}) U^* s.$$

However, computing the Fourier basis for $\mathcal{L}$ is computationally expensive and unstable for large graphs. This motivates the idea for using the Lanczos method for evaluating the matrix function $g(\mathcal{L})$.

In this work, we will restrict our analysis to two graph models: Erdős-Rényi and Sensor graphs. We shall define them as the current implementation in Library [2].

Definition 1.4 (Erdős-Rényi graph)**.** We define the Erdős-Rényi graph $\mathcal{G}_{N,p}$ with N nodes, and probability p, as the graph constructed by randomly connecting two nodes independently from any other edge, with probability p.

Definition 1.5 (Sensor graph)**.** A sensor graph $\mathcal{G}_N$ is the graph constructed by picking N points on the square $[0,1] \times [0,1]$ and connecting points with edges using k nearest neighbours.

Remark 1.6. As currently stating in PyGSP sensor graph source code, definition of Sensor graph changed in 2019, switching to model generation using k-means algorithm. This might give slightly different results than the 2015's article [3] we are trying to reproduce, but we don't investigate this difference.

1.2 The Lanczos method

Definition 1.7 (Krylov subspace)**.** Given a matrix $A \in \mathbb{R}^{N\times N}$ and a vector $b \in R^N$, the Krylov subspace of order l is defined as the set $\mathcal{K}_l(A,b) = \text{Span}\left(b, Ab, A^2b, \dots, A^{l-1}b\right)$. We represent the basis of this subspace with a matrix $V_M = [v_1, \dots, v_M] \in \mathbb{R}^{N\times M}$.

From the Arnoldi relation,

$$AV_l = V_l H_l + h_{l+1,l} v_{l+1} e_l^*, \qquad H_l = V_l^* A V_l = \begin{bmatrix} h_{11} & \dots & \dots & h_{1,l} \\ h_{21} & h_{22} & & \vdots \\ & \ddots & \ddots & \vdots \\ & & h_{l,l-1} & h_{l,l}. \end{bmatrix},$$

we can derive the Lanczos method for a symmetric, positively defined matrix A. Setting

$$\alpha_l := h_{l,l}, \qquad \beta_l := h_{l+1,l}.$$

From our hypothesis on A, it follows that the matrix H_l, which is both upper and lower Hessenberg matrix, is also symmetric; in other words, it is tridiagonal—thus we rename it T_l. Hence the Arnoldi relation takes the form:

$$AV_l = V_l \alpha_l + \beta_l v_{l+1} e_l^T, \qquad T_l = V_l^\top A V_l = \begin{bmatrix} \alpha_1 & \beta_1 & & \\ \beta_1 & \ddots & \ddots & \\ & \ddots & \ddots & \beta_{l-1} \\ & & \beta_{l-1} & \alpha_l \end{bmatrix}.$$

.

We now introduce an useful result for evaluating a matrix function using the Arnoldi projection.

Proposition 1.8. *Consider the filter $g : [0, \lambda_{max}] \to \mathbb{R}$ and a signal vector $s \in \mathbb{R}^N$. Then the Lanczos method approximates $g(\mathcal{L})s$ with:* [1]

$$g(\mathcal{L})s \approx V_M \|s\|_2 g(T_M)e_1 =: g_M. \tag{1}$$

Numerically, we often compute the eigendecomposition (Λ, U) from T_M, and then substitute $g(T_M) = Ug(\Lambda)U^\top$ in the equation above.

1.3 Algorithm

We begin by defining some quantities that will be useful for describing our implementation of the algorithms.

Definition 1.9. (Errors) We define the Lanczos *iteration error* as $\|g_{M+j} - g_M\|_2$, where j is small, and the *true error* as $\|e_M\|_2 = \|g(\mathcal{L})s - g_M\|_2$.

We also define error bounds:

Definition 1.10. We indicate with $\tau =: \tau_{\text{STOP}}$ the algorithm stopping threshold, that is when $\|g_{M+3} - g_M\|_2 < \tau$, and with $\delta =: \delta_{\text{ORTH}}$ the orthogonality threshold, that is when $\|V^\top V - I\|_2 < \delta$.

In floating point arithmetic the basis V_M suffers from loss of orthogonality, and orthogonalizing the incoming vector w only against v_j, v_{j-1} in insufficient, for example if the input matrix A is badly conditioned. Hence, the need to reorthogonalize the new vector w against all the previous basis vectors using some steps of the Grahm-Schmidt process. To do so, we implement three algorithms, which we refer as Lanczos in Algorithm 1 and Arnoldi in Algorithm 5, which we implement following [4] and [3]. These two algorithms are often called by the orthogonalization selection in Algorithm 4, and include a stopping criteria based on matrix function convergence. Before describing the algorithms, we explain how g_M is evaluated in the lanczos process using Equation 1. Recall that $g_M := \|b\|_2 V_M g(T_M)e_1$, where $g(T_M)$ is computed by diagonalizing T_M: because T_M is symmetric, real and tridiagonal, computing its eigendecomposition is stable and efficient; indeed the structure of T_M allows us to use a divide and conquer algorithm for eigenvalues, and we'll use `eigh_tridiagonal` routine from Scipy which has cost $O(N^2)$.

1.3.1 Computational cost

Here we overview computational cost of the algorithm that we tested, for an input sparse matrix A, where $\text{nnz}(A)$ is the number of non-zero elements of A:

- **Lanczos:** Computes V with partial orthogonalization, with $O(MN + M \cdot \text{nnz}(A))$. Note that the product Av_j dominates the cost, and that $\text{nnz}(A) = N^2$ when A is dense. When we need to evaluate function g at each iteration, such as in Section 2.2, we must consider also the $O(N^2)$ operations to compute $g(\mathcal{L})$ with divide and conquer, thus the total cost in this case becomes $O(MN^2)$.

[1] This results holds in general for any function $f : \Omega \to \mathbb{C}$, where $\Omega \subset \Lambda(A)$.

Algorithm 1 Modified Lanczos method in floating point arithmetic.

Input: Symmetric matrix $A \in \mathbb{R}^{N\times N}$, vector $b \in R^N$, $M \in \mathbb{N}$, function g, tolerance τ.
Output: Basis matrix $V_l = [v_1, \dots, v_l] \in R^{N\times l}$ for $K_l(A, b)$, vectors $\alpha = \{\alpha_1, \dots, \alpha_l\} \in \mathbb{R}^l$, $\beta = [\beta_1, \dots, \beta_{l-1}] \in \mathbb{R}^{l-1}$, where $l \in \{1, \dots, M\}$ is the stopping iteration index.
Initialise $V \leftarrow \mathbf{0}^{N\times M}$, $\alpha \leftarrow \mathbf{0}^N$, and $\beta \leftarrow \mathbf{0}^{N-1}$
for $j = 1, \dots, M-1$ **do**
  $w \leftarrow Av_j$
  $\alpha_j \leftarrow v_j^\top w$
  $\begin{cases} w \leftarrow w - \alpha_j v_j & \text{No reorthogonalization} \\ \textbf{if } j > 0 \textbf{ then } w \leftarrow w - \beta_j v_{j-1} \textbf{ end if} & \\ & \\ w = w - \sum_{i=1}^{j} (w^\top v_i) v_i & \text{Full reorthogonalization} \\ w = w - \sum_{i=1}^{j} (w^\top v_i) v_i & \end{cases}$
  if $j < M$ **then**
    $\beta_j = \|w\|_2$
    if $\beta_j = 0$ **then**
      return $V_j, \alpha_{1:j}, \beta_{1:j-1}$ ▷ — Breakdown —
    else
      if $\|g_j - g_{j-3}\|_2 < \tau$ **then**
        return V_j, α, β ▷ — Breakdown —
      end if
      $v_{j+1} = \frac{w}{\beta_j}$
    end if
  end if
end for
return $V_M, \alpha_{1:M}, \beta_{1:M-1}$.

Algorithm 2 Approximation of $g(\mathcal{L})s$ using Lanczos.

Input: Function g, Matrix T, signal s.
$\lambda, U \leftarrow$ diagonalize(T) ▷ Here λ is the vector of eigenvalues.
$u_1 \leftarrow (U^\top)_{:,1}$
$y \leftarrow \|s\| U(\text{Diag}(g(\lambda))U^\top)e_1 = \|s\| U(g(\lambda) \odot u_1)$
▷ Here $u \odot v$ denotes the element-wise product between two vectors u and v.
return Vy

Algorithm 3 Evaluation of $g(\mathcal{L})s$ using Fourier basis.

Input: Function g, Graph $\mathcal{G}$, signal s.
$\Lambda, U \leftarrow$ Fourier_basis($\mathcal{G}$)
return $Ug(\Lambda)U^\top s$

- **Lanczos with double orthogonalization:** It differs from Lanczos by the computation of $w = w - \sum_{i=1}^{j} v_i(v_i^\top w)$, which can cost up to $O(MN)$. Thus the total cost becomes $O(M^2N + M \cdot \text{nnz}(A))$, and when we also evaluate g it becomes $O(M^2N + MN^2)$.

Algorithm 4 Orthogonalization selection.

Input: $\delta = 10^{-10}$.
$V, T, j \leftarrow$ Lanczos(A,b, classic)
if $\|V^\top V - I\|_2 < \delta$ **then**
    $V, T, j \leftarrow$ Lanczos(A,b, full)
end if

- **Arnoldi with double orthogonalization**: For the orthogonalization it takes $O(M^2N)$, plus the matrix vector product at each step with A, hence again $O(M^2N + M \cdot \text{nnz(A)})$. Cost is higher when evaluating g, due to the fact that matrix H is not perfectly symmetric (it is up to a small epsilon) and not tridiagonal (it has some small perturbations). Hence we must use the Schur-Parlett algorithm to compute the matrix function (which we use in the routine `scipy.linalg.funm`), which adds up to a quite large $O(MN^3)$.

We observe that in Lanczos computing g it not necessary a drawback when A is dense or when few iterations are performed. It should also be noted that these algorithms suffer from a significant $O(MN)$ memory cost for storing V_M.

1.3.2 Why double orthogonalization?

From the experimental setting in Section 2.1, we observed that the orthogonality of the basis V with Arnoldi was worse than Lanczos basis. Double orthogonalization showed good results. One reason could be that the Laplacian matrices that we are considering are badly conditioned. The following example motivates this choice:

Example 1.11. We consider a Erdős-Rényi graph with $N = 1000, M = 200, p = 0.04$. The associated Laplacian $\mathcal{L}$ is terribly conditioned, indeed $k_2(\mathcal{L}) = 2.365029616834158 \times 10^{17}$:

Table 1: Correlation between orthogonalization type and basis orthogonality for Lanczos.

Orthogonalization type	$\|V^\top V - I\|_2$
Partial	$6.7883747792312356 \times 10^{-15}$
Single Gram-Schmidt orthogonalization	$1.7133096895857787 \times 10^{2}$
Double Gram-Schmidt orthogonalization	$6.7883747792312356 \times 10^{-15}$

As we can observe in Table 1, orthogonalizing two times against the previous basis vectors yields to a better orthogonal basis V, while a single Grahm-Schmidt iteration performs worse than standard Lanczos.

1.3.3 Comparing Lanczos with Arnoldi

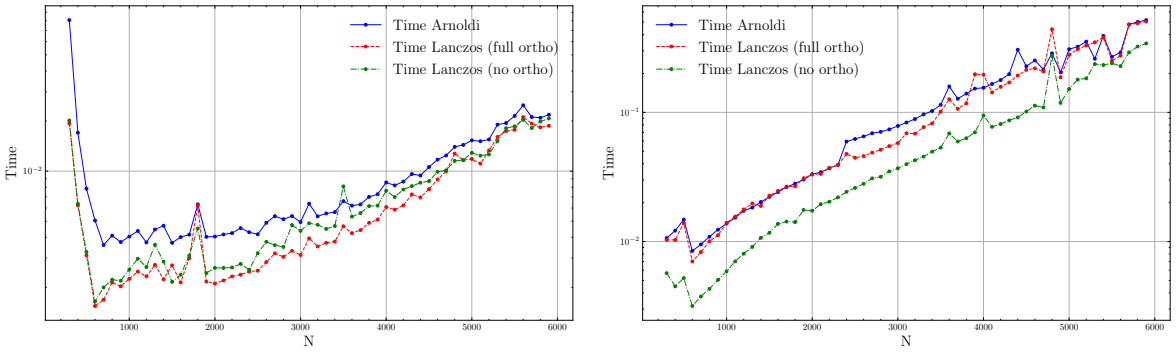
Here we do a short numerical comparison of Lanczos and Arnoldi with double orthogonalization, which should reflect the cost analysis from Section 1.3.1. Algorithm 5 reports our implementation of Arnoldi.

Algorithm 5 Arnoldi with double orthogonalization.

Input: Matrix $A \in R^{N \times N}$, $s \in \mathbb{R}^N$, $M \in \mathbb{N}$, function g, tolerance τ.
Output: Basis matrix $V_l = [v_1, \ldots, v_l] \in R^{N \times l}$ for $K_l(A, b)$, upper-Hessenberg matrix $H \in R^{l \times l}$ where $l \in \{1, \ldots, M\}$ is the stopping iteration index.
$v_1 \leftarrow s/\|s\|_2$
Initialise $V \leftarrow \mathbf{0}^{N \times M}$ and $H \leftarrow \mathbf{0}^{M \times M}$
$V_{:,1} \leftarrow v_1$
for $j = 1 \ldots M - 1$ **do**
    $w_1 \leftarrow Av_j$
    ▷ First orthogonalization
    $h_1 \leftarrow V_j^\top w_1$
    $w_1 \leftarrow w_1 - V_j \cdot h_1$
    $v_{j+1} \leftarrow w_1/\|w_1\|_2$
    ▷ Second orthogonalization pass
    $h_2 \leftarrow V_j^\top v_{j+1}$
    $w_2 \leftarrow v_{j+1} - V_j \cdot h_2$
    $H_{1:j,j} \leftarrow h_1 + h_2$
    $H_{j+1,j} \leftarrow \|w_1\|_2 \|w_2\|_2$

    if $H_{j+1,j} = 0$ **then**
        break ▷ Arnoldi breakdown
    end if
    ▷ Convergence check
    if $j > 3$ **and** $j < M$ **then**
        if $\|g_j - g_{j-3}\|_2 < \tau$ **then**
            return $V_j, H_{1:j,1:j}$
        end if
    end if
    $v_{j+1} \leftarrow w_2/\|w_2|_2$
    $V_{:,j+1} \leftarrow v_{j+1}$
end for
return $V_{:,1:j}, H_{1:j,1:j}$

We consider the function g used in the experiments and $N \in \{300, 400, 500, \ldots, 6000\}$, $M = 200$ and $p = 0.04$. In Figure 2 and Table 2 we see different correlation when running the algorithm with residual break and until M. In the former, described in Figure 2a, Lanczos with full orthogonalization outperforms the other two, in the latter in Figure 2b, curves generally match theoretical results, despite some small cases of the full-orthogonalization variant. One reason for full orthogonalization converging faster than predicted in the complexity analysis, is probably due the spectrum of T_l falling outside the definition interval of g, as we will observe better soon in the experiments in Section 2. In Figure 2a we see Arnoldi slower than the other two, which could be caused by the Schur-Parlett routine bottleneck.



(a) Algorithms stop when $\|g_{l-3} - g_l\|_2 < \tau = 10^{-6}$.

(b) Algorithms run until last index M.

Figure 2: Comparing Lanczos and Arnoldi.

Table 2: Mean and variance of values in Figure 2.

	With τ bound		Run until M	
	Mean	Variance	Mean	Variance
Time Lanczos (no ortho)	0.007253	0.000032	0.078663	0.008244
Time Lanczos (full ortho)	0.006678	0.000032	0.131559	0.019279
Time Arnoldi	0.010095	0.000125	0.137977	0.018646
Stopping index Lanczos (no ortho)	11.614035	25.812657	200	0
Stopping index Lanczos (full ortho)	11.684211	31.005639	200	0
Stopping index Arnoldi	11.684211	31.005639	200	0

2 Experiments

In the following section we generate random Erdős-Rényi and Sensor graphs, as defined in Definitions 1.4 and 1.5, to study Lanczos algorithm's accuracy and performance.

2.1 Comparing global error with projection error

We want to numerically verify the approximation in Equation (1) by relating the residuals in Definition 1.9, which represent the algorithmic error and the ground truth $g(\mathcal{L})$. Let's recall some quantities that will be useful in this section: N is the number of graph nodes and size of the Laplacian $\mathcal{L}$, M is the maximum size of the Krylov subspace, and p the probability of the graph.

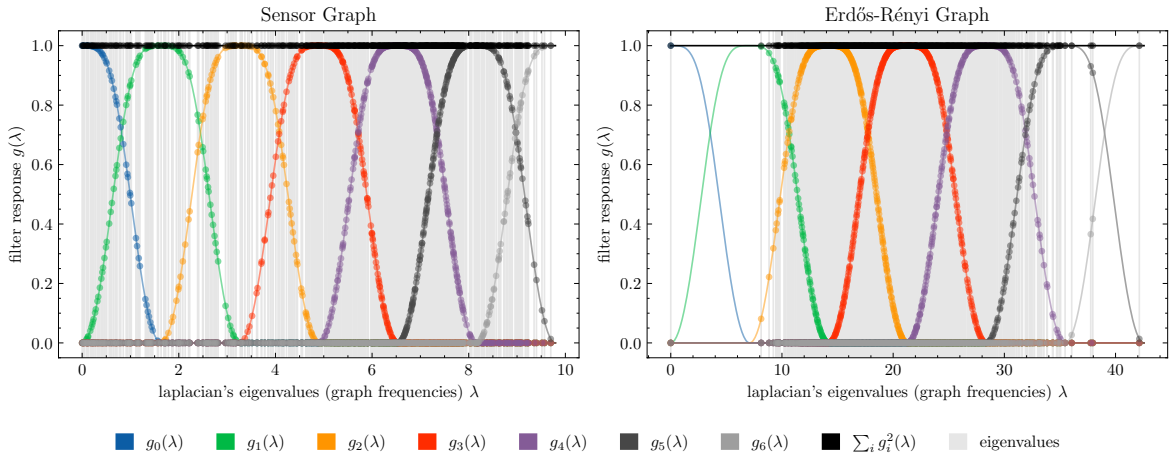
Experiment. We set $(N, M, p) = (500, 200, 0.04)$ and we try to replicate the Experiment 1 in Article [3].

Recall that computing $g(\mathcal{L})s$ is done by computing the Fourier basis for the graph $\mathcal{G}$, which feasible for the small graph that we are considering.

We consider the function $g(t) = \sin(\frac{1}{2}\pi\cos^2(\pi t))\chi_{[-\frac{1}{2},\frac{1}{2}]}$, which we adapt to the graph spectrum, using

the `Itersine` class in Pygsp in Figure 3. Unfortunately, the authors in [3] apply another transformation to this function that adapts g to the Laplacian eigenvalues distrubution, which we were not able to reproduce (despite many trials) as it was not documented both in the article and library. Using the algorithm described in Section 1.3, and computing the errors $\|e_l\|_2$ and $\|g_{l+3} - g_l\|_2$ for every $l = 1 \dots M$, we obtain the plot in Figure 4:

Figure 3: Itersine plot.



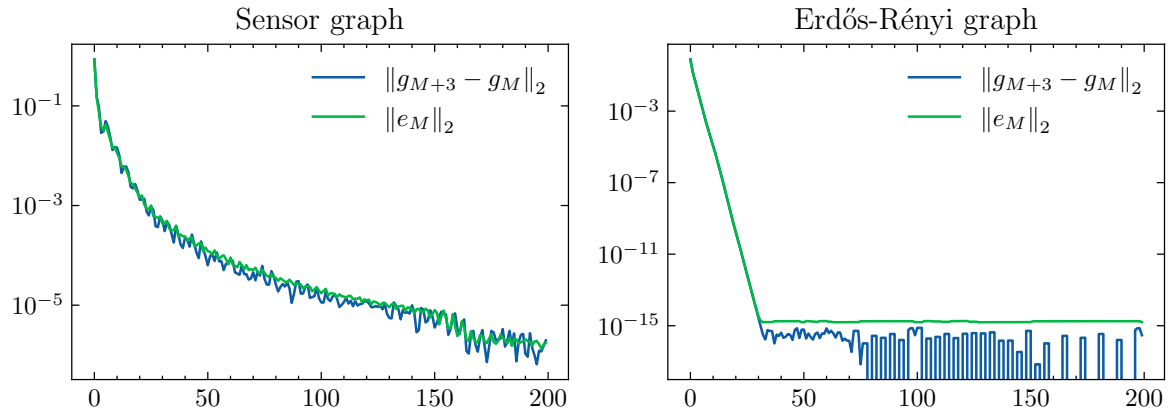


Figure 4: Comparing iteration error with true error.

Let's analyse Figure 4: we immediately observe that the error curves are closely tight, and that their value is decreasing, which shows that the approximation of g gains precision as the basis V_l gets larger. We also see how the eigenvalues of $\mathcal{L}$ and T influence the steepness and smoothness of the curves. Indeed, when eigenvalues fall outside $[-\frac{1}{2}, \frac{1}{2}]$, we see that the curves rapidly reach machine precision: this is unfortunately caused by g not perfectly matching the Laplacian eigenvalues distribution. We also observe evident lack of smoothness in the iteration error curve. We believe that since $g(t) = 0$ for $t \in \mathbb{R} \setminus \left[-\frac{1}{2}, \frac{1}{2}\right]$, the eigenvalues of T tend to jump in and out the interval $[-\frac{1}{2}, \frac{1}{2}]$, causing spikes. Hence, we found that this method is sensible to the eigenvalues distrubution of the Laplacian.

We also found interesting to visualize graphically as in Figure 1 how the signal s is filtered by the algorithm on the graphs used in this experiment, which corresponds to applying a low-pass filter.

2.2 Performance comparison for Erdős-Rényi graphs

In this experiment, we will study the performance correlation for running Lanczos on Erdős-Rényi graphs $\mathcal{G}(N, p)$ and comparing execution time when only N varies and when only p varies.

Remark 2.1 (Role of N and p). Erdős-Rényi graphs, N and p dictate the expected properties of the matrix $\mathcal{L}$, that is dimension and sparsity, respectively. In fact, when N increases, we get expect higher computational cost due to the increased cost of the matrix-vector multiplication $\mathcal{L}v$ and to compute the eigenvalues of T_M, but when p increases, $\mathcal{L}$ becomes more dense, requiring more floating point operations for the algorithm and so increasing runtime. Figure 5 shows how this property on some the matrices we used.

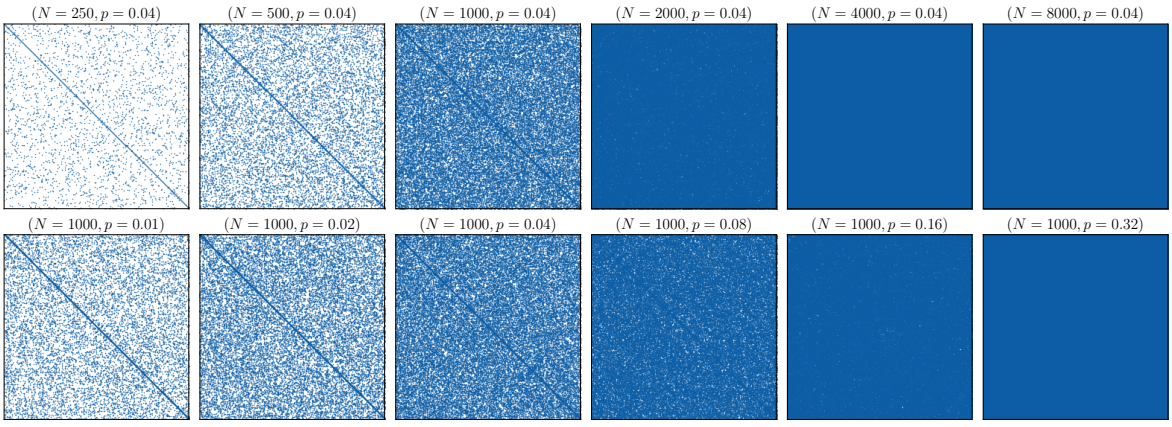
We choose $\delta = 10^{-5}$, as we experimentally observed that smaller thresholds rarely triggered full orthogonalization. As in Algorithm 1 our stopping criteria happens when $\|g_l - g_{l-3}\|_2 < \tau$, at the step l. [2]

Experiment. In the following experiment, we choose generate two sample datasets, letting $n = 6$,:

1. N varies in $\{250, 500, \ldots, 2^n \cdot 250\}$, $\tilde{p} = 0.04$.
2. p varies in $\{0.01, 0.02, \ldots, 2^n \cdot 0.01\}$, $\tilde{N} = 1000$.

With abuse of notation we will often refer to the tuple (N, p) to indicate the columns of the datasets above, for example $(N, p) = (500, 0.02)$, as we are interested in correlating increasing couples of $(N, p) \in \{(250, 0.01), (500, 0.02), \ldots, (8000, 0.32)\}$. We repeat the generation of these graphs 200 times to avoid graph randomness biasing our results.

Figure 5: Plot of the sparsity pattern of the generated matrices, as N, p varies.



For time telemetry, we adopt Python's `time.perf_monitor()` which ensures precise measurement of the execution in Algorithm 4. We aknowledge that this method suffers from executing Lanczos twice but from

[2] Here index goes backwards because matrix T_j would not be defined for $j > l$ at iteration l!

what we saw for some cases in Table 3 it is convenient triggering full orthogonalization with selection as in Algorithm 4. From these first examples, we think it's difficult to precict whether the basis will need or not to be reorthogonalized.

We now show plot of some runs of this experiment for some conditions on τ, δ and the stopping criteria.

2.2.1 Run with selective orthogonalization, $\tau = 10^{-5}$ and $\delta = 10^{-5}$

Looking at Table 4 and 6a, we observe that time is higher for $(N, p) = (250, 0.01)$, and then increases gradually. In the case of N, we know from the theory that when $N \gg M$, the Krylov subspace is approximated better by the algorithm, hence time start growing stably from $N \geq 500$, and we see a significant 4x gap from $N = 4000$ and $N = 8000$. When p varies the matrices become dense and we expect longer computational times, as the matrix may loose orthogonality for dense matrices, but we see the opposite in Table 4. Indeed we see time for $p = 0.01$ more than a decimal of second, much more than the other cases, and then obscillating for other values of p, where the τ bound stops the algorithm early. We believe that this is due the phoenomena we already observed in Section 2.1: the error $g_{l-3} - g_l$ reaches machine precision fast as depicted in Figure 4, because the eigenvalues of the input matrix $\mathcal{L}$, which becomes dense as p grows, start falling outside the domain of g.

In general, in Figure 6a we see a linear growth with few outliers, where times for increasing p result slower than those of N.

In the next Section 2.2.2 we try to run the procedure until the index M to verify how the error bound is influencing performance.

Table 3: Timing the 12 matrices in Figure 5 for just one run, where N varies and p fixed (top), and viceversa (bottom).

N	M	p	Time (s)	Orthogonalization	Breakdown	Stopping index	ε
250	200	0.04	0.025940	Full	Bound	61	10^{-5}
500	200	0.04	0.002009	Partial	Bound	16	10^{-5}
1000	200	0.04	0.001946	Partial	Bound	13	10^{-5}
2000	200	0.04	0.005585	Full	Bound	11	10^{-5}
4000	200	0.04	0.012243	Full	Bound	10	10^{-5}
8000	200	0.04	0.059831	Full	Bound	9	10^{-5}
1000	200	0.01	0.137948	Full	Bound	131	10^{-5}
1000	200	0.02	0.006453	Full	Bound	23	10^{-5}
1000	200	0.04	0.001641	Partial	Bound	13	10^{-5}
1000	200	0.08	0.003160	Full	Bound	11	10^{-5}
1000	200	0.16	0.003718	Full	Bound	10	10^{-5}
1000	200	0.32	0.002654	Partial	Bound	9	10^{-5}

Table 4: Stats when N and p varies (top and bottom).

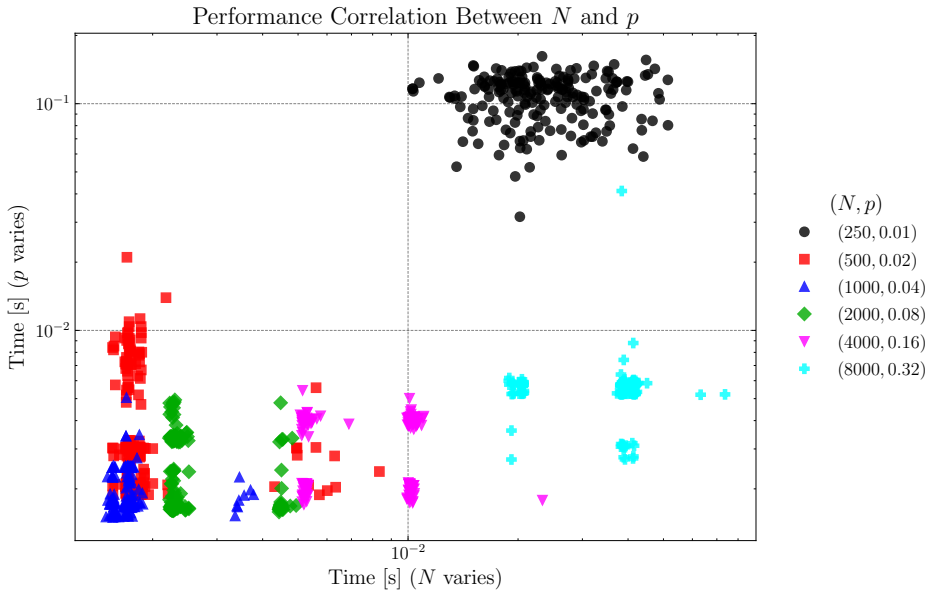
		Time		Stopping index	
N	p	mean	var	mean	var
250	0.04	0.024815	0.000077	61.125000	138.602387
500	0.04	0.001995	0.000001	15.655000	3.031131
1000	0.04	0.001761	0.000000	12.780000	0.212663
2000	0.04	0.002632	0.000001	11.050000	0.047739
4000	0.04	0.008299	0.000007	10.000000	0.000000
8000	0.04	0.036846	0.000070	9.000000	0.000000
p	N				
0.01	1000	0.107721	0.000594	117.410000	111.408945
0.02	1000	0.004034	0.000008	17.865000	14.036960
0.04	1000	0.001913	0.000000	12.770000	0.218191
0.08	1000	0.002411	0.000001	11.000000	0.000000
0.16	1000	0.003342	0.000001	9.840000	0.135075
0.32	1000	0.005640	0.000007	9.000000	0.000000

2.2.2 Run with selective orthogonalization, no error bounds and $\delta = 10^{-5}$

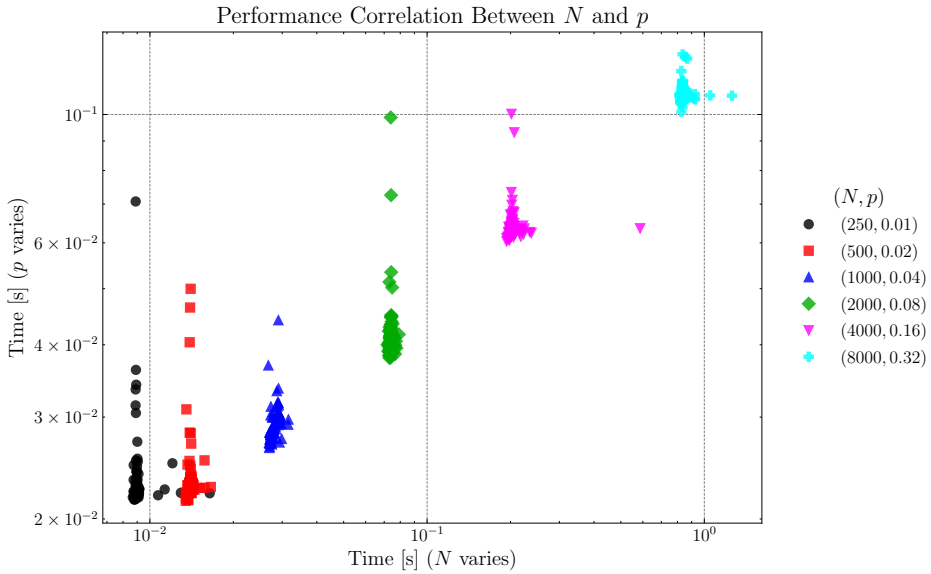
In Figure 6b and Table 5 we observe an almost-linear correlation, with few outliers which could be caused by the fact that full orthogonalization is triggered in some cases, causing two runs of the algorithm. We believe that we could see a similar pattern (or more correlation) in case the algorithm runs with a different g.

3 Conclusions & future work

To conclude, we observed how stategic bound and orthogonalization techniques can improve performance trough early breaks, especially for large and dense matrices. However, due the fact that the funciton was piecewise continuous on the spectrum values we observed that these bound can start to be unrelyiable and exit the algorithm too early, when the Krylov subspace may be not well-formed. We didn't address that the high memory cost of $O(NM)$ of Lanczos could be avoided by using a two-step implementation. Further, because full orthogonalization was widely used in the second experiment, we could implement selective reorthogonalization as described in [4], but both methods fall behind the objective of this work. Another improvement, which we already discussed, would be to implement a working `WarpedTranslates` class in Library [2].



(a) Correlation stopping with τ bound.



(b) Correlation when running until M.

Figure 6: Log-log plot of 200 iterations of the comparison experiment.

Table 5: Stats when N and p varies (top and bottom) without bound.

		Time		Stopping index	
		mean	var	mean	var
N	p				
250	0.040000	0.009062	0.000001	200	0
500	0.040000	0.014043	0.000000	200	0
1000	0.040000	0.028319	0.000001	200	0
2000	0.040000	0.074183	0.000001	200	0
4000	0.040000	0.204908	0.000764	200	0
8000	0.040000	0.841120	0.001346	200	0
p	N				
0.01	1000	0.022907	0.000015	200	0
0.02	1000	0.023276	0.000009	200	0
0.04	1000	0.029179	0.000003	200	0
0.08	1000	0.041019	0.000025	200	0
0.16	1000	0.063665	0.000014	200	0
0.32	1000	0.108326	0.000008	200	0

References

[1] Ulrike Von Luxburg. A tutorial on spectral clustering. *Statistics and computing*, 17(4):395–416, 2007.

[2] Michaël Defferrard, Lionel Martin, Rodrigo Pena, and Nathanaël Perraudin. Pygsp: Graph signal processing in python.

[3] Ana Susnjara, Nathanael Perraudin, Daniel Kressner, and Pierre Vandergheynst. Accelerated filtering on graphs using lanczos method. 2015.

[4] James W Demmel. *Applied numerical linear algebra*. SIAM, 1997.